EE6003 Embedded Systems

Coursework Master Plan

Advanced Industrial Robotic Arm using RF Wireless Communication

Roadmap to a 1st-class submission

Prepared for: k — third-year electromechanical engineering

Deadline: 25 June 2026 | Group size: 3 | Hardware: Arduino + HC-05

1. Grade Strategy: How to Break the 60% Ceiling

The single most important sentence in your brief is this:

> *"Students who correctly implement all core coursework requirements may achieve marks up to a maximum of 60% (upper 2:1 threshold). Marks above 60% will be awarded only where students demonstrate advanced or novel embedded systems features."*

This means a perfectly built arm that ticks every core requirement is still capped at 60%. To reach a 1st (70%+) or 1st+ (80%+), you must deliberately layer on advanced embedded-systems features AND explicitly label them as such in your report. The rubric lists the advanced features by name — they are the scoring lever.

1.1 The advanced feature checklist

Every feature below is named directly in the rubric or the brief. Your target is to demonstrably implement as many as possible, even if each is small, and to explicitly section them in the report as 'advanced'.

- **Real-time / FreeRTOS task scheduling:** Use the ArduinoFreeRTOS library (AVR port). Split firmware into tasks: joystick sampling, packet TX, motor control, feedback PID, safety watchdog. Documents as 'deterministic multi-task real-time execution'.
- **Interrupt-Service-Routine (ISR) driven design:** Use pin-change interrupts for the emergency-stop button, timer interrupts for PID loop at a fixed rate (e.g. 100 Hz), and ADC free-running interrupts for potentiometer sampling. This is cheap to implement and huge for the rubric.
- **Finite State Machine (FSM):** Implement an explicit state machine for the autonomous mode (IDLE → SCAN → DETECT → CLASSIFY → PICK → PLACE → COUNT_CHECK). Draw the state diagram. Also use a small FSM for mode-switching between manual and autonomous.
- **Closed-loop position control:** Implement a PID controller on every joint using potentiometer feedback. Tune and report Kp/Ki/Kd values and step-response plots — that's directly a 1st-class testing deliverable.
- **Custom wireless protocol design:** Design your own packet structure with header, joint IDs, payload, checksum (CRC-8 or XOR), and sequence number. Specify baud rate, acknowledgement scheme, and reconnection logic. Arduino + HC-05 can absolutely do this.
- **Dual / redundant wireless (optional, very strong):** Consider adding an nRF24L01+ module alongside HC-05 so you can compare protocols, or use it as a fall-back. Two protocols compared = a dedicated report sub-section.
- **Robustness / error handling:** Packet loss detection, checksum failures logged, command timeouts that park the arm in a safe pose, joint-range soft-limits in firmware.
- **Safety modes:** Physical e-stop button wired to an interrupt, watchdog timer that resets the MCU if the main loop hangs, current-limit sensing on motors (optional).
- **Power management:** ATmega sleep modes when idle, disabling ADC/Bluetooth when not needed, measured current profile in active vs idle vs sleep. Report a power budget table.
- **Sensor fusion or advanced sensing:** Colour detection via TCS3200/TCS34725 module, IR proximity sensor for the autonomous scan step. Possibly add a load cell on the gripper.

1.2 Mapping features to rubric marks

Cross-check before you submit: every rubric row below must contain at least one 'advanced' claim clearly labelled in the text.

Rubric row	Marks	Your advanced hook (what to write in the report)
Abstract / Intro / Lit Review	10	Critical comparison of BLE vs Classic BT vs Zigbee vs nRF, with a trade-off table and references. Discussion of industrial interference (2.4 GHz congestion, Wi-Fi coexistence).
Embedded System Design & Architecture	10	Block diagram showing two units + power domain + wireless link + feedback loop. Justified MCU selection table. Mode-switching state diagram.
Hardware Development & Sensor Integration	20	Schematic with decoupling caps, pull-ups, protection diodes. Discussion of ADC noise mitigation (RC filter on pot wipers, averaging). Current budget and separate servo/logic power rails.
Software Development & Bluetooth Comm.	15	FreeRTOS task diagram, FSM diagrams, custom packet format table, ISR usage list, PID tuning plots. Code snippets with line-level comments.
Robotic Arm Design & 3D Printing	10	Own CAD from scratch (or heavily reworked), print-parameter table (infill, layer height, material), FDM vs SLA justification, DFM tweaks you made after first print.
Testing & Performance Evaluation	20	Quantitative: latency histograms, repeatability (N trials, sigma), packet-loss sweep vs distance, power consumption by mode. Before/after optimisation data.
Teamwork & Professional Feedback	10	Kept log of task allocation, weekly stand-up notes, Gantt chart showing individual work-streams. Peer feedback submitted independently.
Report Writing & Documentation	5	IEEE template, consistent citations, figure captions, zero grammar issues. Demonstration video that maps directly onto report sections.

2. Critical Risks and Mitigations

2.1 The Arduino + HC-05 ceiling risk

Arduino Uno is almost certainly too small — you will run out of pins, timers, and SRAM once you add 4 servos, 4 potentiometers, a Bluetooth UART, an IR sensor, and a colour sensor. Use an Arduino Mega 2560 for the arm (plenty of I/O, 4 hardware UARTs, more flash). The transmitter can stay on an Uno/Nano.

HC-05 is Classic Bluetooth SPP — it works, but the rubric rewards 'advanced wireless' thinking. Mitigations:

- Design a custom packet protocol on top of HC-05's SPP tunnel — header, CRC, sequence, ACK. That is 'advanced communication protocol design' even if the radio layer is boring.
- Optional: add an nRF24L01+ as a second channel and present a comparison. Costs £3, gives you an entire report sub-section.
- Run FreeRTOS on the Mega using the ArduinoFreeRTOS library — this unlocks the real-time / deterministic claim.

2.2 The CAD originality risk

The brief explicitly requires 'a SolidWorks (or equivalent) design file of the robotic arm'. Printing someone else's STL files without modification will almost certainly lose most of the 10 marks for this section and could be flagged for plagiarism. Mitigations in order of preference:

1. Best: rebuild the arm from scratch in SolidWorks or Fusion 360 using the YouTube design as a reference only. Change link lengths, joint layouts, gripper geometry. You will learn a lot and justify design choices in the report.
2. Acceptable: import the STL meshes into CAD, then design original brackets, a custom controller enclosure, a custom gripper, potentiometer mounts, and a base plate. At minimum 50% of printed parts should be your own.
3. Risky: print the YouTube files unchanged but add bolt-on sensor brackets and a controller case. You must then credit the original source and frame your contribution as integration/modification — marks limited but defensible.

2.3 Timeline risk

10 weeks including 3D printing, firmware debugging, and report writing is tight. The main killers are: (a) waiting for 3D-printed parts to finish between iterations; (b) motor stall, servo jitter, and ADC noise issues that only show up once everything is assembled; (c) leaving the report until the last week and running into the 10% word-count overflow cap. The Gantt schedule in Section 5 is designed with buffers to absorb these.

2.4 Academic integrity and plagiarism

Zero tolerance per the brief. Practical implications:

- Every code snippet copied from an example or library must be attributed in a comment and in the bibliography.
- Every figure or block diagram adapted from elsewhere must cite the source.
- Run the final report through Turnitin/the Institute's similarity checker before submitting.
- Keep a shared 'sources' spreadsheet — IEEE reference number, URL, retrieval date, which section uses it.

3. Bill of Materials

The list below assumes Arduino + HC-05 on both units, a 4-DOF + gripper arm with potentiometer feedback on each joint, and a colour+IR autonomous sort task. Expect £80–£130 total if nothing is already in the lab.

3.1 Transmitter (handheld controller)

Qty	Part	Purpose / Notes
1	Arduino Nano or Uno	ATmega328P, small and portable
1	HC-05 Bluetooth module	Configure as Master to pair with the arm's HC-05 Slave
2	Analog thumbsticks (KY-023 or similar)	4 axes total — covers shoulder, elbow, base rotation, wrist (or gripper)
1	Push-button (momentary)	Mode switch (Manual ↔ Autonomous)
1	Push-button (latching red)	Emergency-stop — wired to a pin-change interrupt, NOT just polled
1	9 V battery + buck to 5 V, or 2×18650 + buck	Power; measure current draw for the report's power budget
—	Perfboard, enclosure, cables	For a clean prototype — ideally your own 3D-printed enclosure

3.2 Receiver (robotic arm)

Qty	Part	Purpose / Notes
1	Arduino Mega 2560	You need the extra I/O, timers and SRAM for FreeRTOS + 4 servos + sensors
1	HC-05 Bluetooth module	Slave role
4	MG996R or DS3225 servos	Higher torque for base and shoulder; MG90S is fine for wrist/gripper
1	SG90 servo for gripper (or mini)	Or a geared DC motor if using parallel gripper
4	10 kΩ linear potentiometers	Glued/geared to each joint for absolute-position feedback. Use RC low-pass on each wiper.
1	TCS34725 colour sensor (I2C)	For the colour-sort autonomous task; library available
1	Short-range IR sensor (e.g. GP2Y0A21, or sharp IR / TCRT5000)	Detects object presence in pickup area
1	5 V / 5–6 A BEC or SMPS	Servos need their own rail, separate from the MCU 5 V
1	7.4 V Li-ion pack or 5 V 5 A mains adapter	Power source — mains is simpler for benchtop testing
—	Decoupling caps (100 nF + 470 µF per servo rail), common ground	Critical for servo-induced noise — document in hardware section
—	Pre-crimped servo extensions, ribbon cable, heat-shrink	Clean wiring = better demo video

3.3 Software and tools

- Arduino IDE or PlatformIO (PlatformIO strongly recommended for version control and library management)
- ArduinoFreeRTOS library (Phillip Stevens port) — unlocks FreeRTOS on ATmega
- PID library (br3ttb/Arduino-PID) OR write your own PID in ~20 lines for extra credit
- SolidWorks, Fusion 360, or NX (whichever your institute provides) for CAD
- Cura or PrusaSlicer for 3D print preparation
- KiCad or Fritzing for schematics; KiCad is the professional choice and plots well in the report
- Draw.io or Lucidchart for block and state diagrams
- Git + GitHub (private repo) — also serves as evidence of individual contribution via commit logs
- Overleaf with the IEEE conference template for the report
- Logic analyser or oscilloscope for the testing section — even a cheap Saleae clone is invaluable for latency measurements

4. Team Roles and Individual Contribution Mapping

The brief lists eight contribution areas and expects the report to clearly identify who did what. With 3 people, each person should own 2–3 areas and contribute to a fourth. As the flexible/lead member, you take the integration role — this is the strongest position because you touch every subsystem and can claim breadth in the peer-reviewed individual contribution.

4.1 Suggested allocation

Member A — Hardware Lead	Member B — Software Lead	Member C (YOU) — Integration & Testing Lead
• Hardware Development & Sensor Integration (main) • Embedded System Design & Architecture (shared) • Robotic Arm Design & 3D Printing (shared if that's their strength) • Schematic + PCB/perfboard layout • Power budget and protection circuits • Owns the hardware section of the report	• Software Development & Bluetooth Communication (main) • Embedded System Design & Architecture (shared) • FreeRTOS task design, FSM, PID, packet protocol • Firmware documentation and code comments • Owns the software section of the report	• Testing & Performance Evaluation (main) • Robotic Arm Design & 3D Printing (main) — plays to your electromechanical background • Abstract / Introduction / Literature Review (main) • Report Writing & Documentation (main) • Project management: Gantt chart, weekly meetings, integration • Demonstration video editing

This gives you (Member C) ownership of four sections (Testing, Arm Design, Lit Review, Report) plus PM oversight — which aligns with 'flexible / overall lead'. It also gives you the most marks on paper: 20 (testing) + 10 (arm design) + 10 (lit review) + 5 (report writing) = 45 marks primarily yours, with the rest shared.

4.2 Evidence of individual contribution — keep from day one

The peer-feedback section (10 marks) requires each student to justify their own contribution. Start logging evidence in week 1, not week 9:

- Shared Git repository with per-person branches — commit history is the cleanest possible evidence
- Weekly stand-up notes (even 5 bullet points) saved to a shared Notion/Google Doc
- Screenshots of CAD and schematic files with file-owner names visible
- Named figures in the report — every figure caption should credit the creator, e.g. 'Fig. 4. Wireless packet structure (designed by Member B)'
- A contribution table in the report appendix — which sections of which code files each person wrote, with % ownership

5. 10-Week Timeline

Deadline: 25 June 2026 (Thursday). Working backwards from that, with today's date being 16 April 2026, you have roughly 10 calendar weeks. The schedule below assumes you spend 4–8 hours per person per week; scale up the earlier weeks if you can.

Week	Theme	Deliverables / tasks
Week 1 (16–22 Apr)	Kick-off & planning	Finalise team roles. Order all BOM parts (do this today — lead times kill projects). Set up GitHub repo, Overleaf project, shared drive. Read this plan end-to-end. Split lit-review topics between members. Download IEEE template.
Week 2 (23–29 Apr)	Literature review & architecture	Each member writes their assigned lit-review topic (~400 words). Draft full system block diagram. Decide on packet format. Set up Arduino IDE/PlatformIO on all machines. Run 'blink' on both Arduinos. Test HC-05 pairing with a terminal app.
Week 3 (30 Apr–6 May)	CAD & first print	Member C owns CAD this week — model the base, shoulder, elbow, wrist and gripper from scratch (or heavily modified). Start printing the base and shoulder. Members A/B breadboard the transmitter and get joystick ADC values streaming over serial.
Week 4 (7–13 May)	First wireless data	Transmitter → Receiver 'hello packet' over HC-05 with your custom packet format and CRC. Member A assembles first printed joints and mounts pots. Member B implements the FSM skeleton (IDLE, MANUAL, AUTONOMOUS, SAFE). Record a short video of joystick values appearing on the arm's serial monitor — evidence.
Week 5 (14–20 May)	Manual mode working	All 4 joints + gripper move from joystick input with live potentiometer feedback. Add FreeRTOS: joystick task (50 Hz), TX task (50 Hz), RX task (ISR-driven), motor task (100 Hz), feedback task (100 Hz), safety task (10 Hz). Basic PID on each joint — just P term first, then PI.
Week 6 (21–27 May)	Autonomous mode + colour sort	Implement the Sense-Think-Act FSM. IR triggers scan, colour sensor reads, state transitions to pick/place, counter increments. Demonstrate sorting 5 objects. This is the biggest-risk week — start early.
Week 7 (28 May–3 Jun)	Testing & optimisation	Formal test campaign: latency, repeatability (N=30 runs per joint), packet-loss vs distance, current draw by mode. Tune PID gains. Add power management (sleep in idle). Fix any wire routing / mechanical issues. Begin writing testing section of report.
Week 8 (4–10 Jun)	Report draft 1	Full draft of every section. Target 4500 words — do not over-write, the 10% overflow cap is brutal. Make sure every advanced feature is explicitly labelled. Insert all figures, schematics, photos, code snippets. Peer-review each other's sections.
Week 9 (11–17 Jun)	Polish, video, rehearsal	Final bug fixes and any last performance tuning. Record the demonstration video — cover every rubric row explicitly. Each member rehearses the questions they'll be asked about their contributions. Proof-read report; run through Turnitin early.

Week	Theme	Deliverables / tasks
Week 10 (18–24 Jun)	Submit	Final polish on report. Each member writes their individual peer-feedback statements and submits independently to Canvas. Package code/videos/CAD into the supplementary zip. Submit by 23:59 on 25 June at the latest — aim for 48 hours earlier to absorb upload issues.

6. Weekly Documentation Checklist

The quickest way to lose marks on a project like this is to build a great prototype and then have to reverse-engineer the report from memory in week 9. Instead, document AS YOU BUILD. Every week, each member saves:

- A dated note (one paragraph) of what they worked on and what worked / didn't work
- Any photos of breadboards, assemblies, or prints — keep ALL of them, even failed prints (they're gold for the report's design-iteration story)
- Screenshots of CAD iterations, oscilloscope captures, serial monitor output
- Raw CSV logs from any measurements (latency, PID response, current draw) — in a testing/ folder in the repo
- Any references cited that week — into the shared bibliography spreadsheet
- Commit messages in Git that describe the 'why' not just the 'what'

These artefacts become your figures, your evidence, and your report's technical depth. Without them, you'll be writing generic prose in week 9.

7. System Architecture

7.1 Block diagram — what to draw

The rubric explicitly rewards 'highly detailed system architecture'. Your block diagram must show, at minimum:

- Transmitter power rail(s) → MCU (Nano/Uno) with labelled pins for joystick ADC, mode-button GPIO, e-stop interrupt
- Transmitter MCU → HC-05 (Master) via UART at a stated baud rate (e.g. 38400)
- Wireless link labelled with protocol (Bluetooth 2.0 SPP), 2.4 GHz, range, and the custom packet structure name
- Receiver HC-05 (Slave) → Arduino Mega via Serial1
- Mega → 4× servos (PWM pins) + 1× gripper servo
- Mega ← 4× potentiometers (ADC, with RC filter shown)
- Mega → TCS34725 (I^2C) and IR sensor (analogue or digital)
- Separate 5–6 V servo power rail with a common GND back to the Mega's GND — this is important and should be drawn explicitly
- Safety line: e-stop button → interrupt pin; watchdog block on the Mega

7.2 Mode control-flow diagram

Draw a second small state diagram for mode switching: IDLE → MANUAL (on joystick movement) → AUTONOMOUS (on button press) → SAFE (on e-stop or comms loss) → IDLE (on reset). Show clearly how a SAFE-state transition parks the arm at a known pose before halting motors. This demonstrates 'safe and stable mode switching' from Objective 7.

8. Software Design

8.1 FreeRTOS task breakdown

Run the following tasks on the Arduino Mega using ArduinoFreeRTOS. Table columns: task, period, priority, job.

Task	Period	Priority	Job
RX_Task	event-driven (ISR-fed queue)	4 (high)	Reads bytes from HC-05 UART into a ring buffer, frames packets, validates CRC, pushes decoded commands onto cmdQueue.
Feedback_Task	10 ms (100 Hz)	3	Samples 4 potentiometers via ADC, applies moving-average filter, updates shared jointState struct guarded by mutex.
Motor_Task	10 ms (100 Hz)	3	Runs PID on each joint using (target − measured), writes servo PWM. Honours joint soft-limits.
Mode_FSM_Task	20 ms (50 Hz)	2	Runs the mode FSM (manual/autonomous/safe) and the autonomous Sense-Think-Act sub-FSM.
Telemetry_Task	100 ms (10 Hz)	1 (low)	Sends joint positions, mode, and error counters back to the transmitter (optional but great for the demo and testing).
Safety_Task	50 ms (20 Hz)	5 (highest)	Checks watchdog, comms-timeout counter, joint-limit violations; if tripped forces SAFE mode and parks motors.

This single table, if you implement it, earns you almost every 'advanced' claim in the software rubric row simultaneously: real-time, deterministic timing, ISR-driven, FSM, robust error handling.

8.2 Wireless packet format (example)

Design your own — but here is a sound starting point with an 8-byte packet over HC-05 SPP. Document this exact table in the report.

Offset	Field	Size	Meaning
0	START	1 B	Sync byte 0xAA
1	SEQ	1 B	Rolling sequence number 0–255 — used to detect drops and reordering
2	CMD	1 B	Command ID: 0x01 joint target, 0x02 gripper, 0x03 mode switch, 0x04 e-stop, 0xF0 telemetry-request
3	JOINT	1 B	Joint ID (0–4)
4–5	VALUE	2 B	Target angle or gripper position, uint16 little-endian
6	CRC8	1 B	CRC-8 (poly 0x07) over bytes 0–5
7	END	1 B	Sync byte 0x55

Features to highlight in the report: start/end framing for robustness against stream desync, sequence number for loss detection, CRC-8 for integrity, compact 8-byte format for low latency, separate CMD/JOINT decoupling so you can add commands without reshaping the protocol.

8.3 Autonomous-mode FSM (Sense-Think-Act)

States and transitions for the sort task. Draw this as a state-diagram figure in the report.

- IDLE → SCAN: on 'start autonomous' command
- SCAN: sweep base slowly; on IR threshold crossed → DETECTED

- DETECTED → CLASSIFY: stop base, read colour sensor, decide target bin
- CLASSIFY → PICK: move arm above object, close gripper
- PICK → PLACE: move over target bin, open gripper
- PLACE → COUNT_CHECK: increment counter; if counter ≥ 5 → DONE, else → IDLE
- Any state → SAFE on e-stop or comms-loss

8.4 Closed-loop control (PID)

Even basic servos have compliance — under load the actual joint angle drifts from the commanded angle. Reading the potentiometer and closing a PID loop around it is a direct objective of the brief (Objective 4). Implementation notes:

- Start with P-only, then add I to eliminate steady-state error, then D to damp overshoot
- Sample rate 100 Hz is plenty for servo-speed joints
- For the report: plot step response (45° to 90°) before and after tuning — huge testing-section win
- Document tuning method (manual Ziegler-Nichols, or trial+error) and the final gain table

9. Hardware Design Notes

9.1 Servo power — THE common failure mode

Powering servos from the Arduino's 5 V regulator will brown-out the MCU as soon as a servo stalls. Use a dedicated 5–6 V supply (BEC or separate SMPS) rated for at least 2× the sum of stall currents, with 470 µF bulk capacitance near the servo connectors and a 100 nF ceramic at each servo's 5 V pin. Common GND back to the Arduino. Document this on the schematic — it is textbook 'hardware robustness'.

9.2 Potentiometer noise

Servo PWM kicks dirt onto the ADC rail. Mitigate with (a) a 1 kΩ series resistor + 100 nF cap at each pot wiper forming a low-pass filter, (b) software averaging of 8–16 ADC samples per reading, (c) separate analogue ground if you can manage it. This is exactly what the rubric means by 'noise handling'.

9.3 Safety / protection

- Series diode + polyfuse on battery input to prevent reverse-polarity damage
- Latching physical e-stop button cutting servo power directly (belt) AND wired to an MCU interrupt (braces)
- Soft joint limits in firmware — stop PWM within the mechanical range
- Watchdog timer (wdt_enable in avr/wdt.h) set to 500 ms, kicked in the main FreeRTOS idle task
- Mechanical e-stop position should be reachable by the demonstrator's non-dominant hand during the video

9.4 Schematic tooling

Do the schematic in KiCad, not Fritzing. KiCad exports cleaner PDFs that look professional in the report. Make TWO schematics — one for the transmitter, one for the receiver — with a consistent title block that includes names, date, revision.

10. CAD and 3D Printing Plan

10.1 Design targets

- Base: ≥180° of rotation, room to mount the base pot to the shaft
- Shoulder & elbow: 0–120° each, reinforced joints with bearing provision
- Wrist: 180°, lightweight
- Gripper: parallel or pincer, ≥40 mm opening, rubberised grip face (nitrile glove scrap or TPU)
- Integrated potentiometer mounts on each joint (this is originality)
- Cable channels through each link — avoids trailing wires in the demo video

10.2 Print parameters to document

Parameter	Recommended / Notes
Material	PLA+ for most parts, PETG for structural parts subject to heat/stress, TPU for gripper pads
Layer height	0.2 mm default; 0.15 mm for gripper and gear features

Parameter	Recommended / Notes
Infill	25% gyroid for most parts; 40–50% for shoulder and base where torque is highest
Walls	4 perimeters on load-bearing parts
Orientation	Long axis vertical for joints to keep layer-lines perpendicular to load
Post-processing	M3 brass inserts (heat-set) for all threaded joints — hugely increases durability and shows professional fabrication

Take a picture of every print — successful AND failed — and of the brass-insert installation. The rubric explicitly rewards 'evidence of improvements'; a before-and-after photo of a redesigned gripper beats a perfect-first-time part every time in a report.

11. Testing and Performance Evaluation Plan

This section is worth 20 marks — tied for the largest. The difference between 2:1 and 1st here is: 2:1 reports say 'the arm worked'; 1st reports have tables and graphs of quantitative data. Plan your tests up-front so you capture data as you build.

11.1 Tests to run and data to log

Test	Method	Data captured	Report figure
End-to-end latency	Send a joint-target packet at time t0 (log on TX serial); log arrival + decode time on RX serial; measure with millis() on both sides synced by a broadcast timestamp	Histogram of 500 samples	Latency histogram
Packet loss vs distance	Transmit 1000 sequenced packets at 1, 3, 5, 10 m; count missed SEQ on RX	CSV: distance vs loss%	Loss curve
Joint repeatability	Command the shoulder to go 0°→90°→0° 30 times, log pot reading at each endpoint	Mean + σ per endpoint	Repeatability table
PID step response	Command a 45°→90° step, log target + measured at 100 Hz for 2 s	Time-domain plot	Pre-tune vs post-tune plots
Current draw per mode	Measure with a bench PSU or INA219; log idle, manual-active, autonomous-active, sleep	Table + bar chart	Power budget
Autonomous sort success rate	Run the sort task 20 times with 5 coloured objects each; log successes and failure modes	Success rate + failure taxonomy	Sort performance
Robustness (fault injection)	Disconnect HC-05 mid-motion; confirm arm parks in SAFE pose; verify watchdog recovery	Pass/fail + recovery time	Safety test results

11.2 Testing-driven improvement narrative

The rubric says 'testing used to guide optimisation and system refinement' is a 1st-class hallmark. Structure your testing section as: (a) initial measurements, (b) problem identified,

(c) what we changed, (d) re-measurement showing improvement. Examples of cause-and-effect pairs you can engineer:

- Latency high → increased baud rate OR reduced payload → latency halved
- Joint oscillates → D-term tuned → oscillation damped
- Pot noisy → added RC filter + 8-sample average → σ dropped from X to Y
- Sort accuracy low → added confirmation sample in colour state → accuracy up

12. Literature Review Plan

Ten marks, and a pure writing task that can be done in parallel with hardware. Split among all three team members. Each sub-topic needs 150–300 words with 3–5 IEEE citations.

12.1 Topics to cover

4. Industrial automation and robotic manipulators — size of market, typical DOF, pick-and-place applications (SCARA, articulated)
5. Wireless protocols comparison — Bluetooth Classic SPP, BLE, Zigbee, Wi-Fi, nRF24L01+, LoRa. Table of: range, throughput, latency, power, cost, interference behaviour in an industrial (2.4 GHz congested) environment
6. Closed-loop feedback in robotic joints — potentiometers vs optical encoders vs magnetic encoders vs absolute encoders; accuracy, cost, reliability
7. PID control fundamentals and tuning methods (Ziegler-Nichols, manual, auto-tune)
8. Finite State Machines in embedded systems — benefits for safety-critical and real-time firmware
9. Real-time operating systems on microcontrollers — FreeRTOS architecture, task priorities, mutexes, ISRs; comparison with bare-metal super-loop
10. Additive manufacturing for mechanical prototypes — FDM material mechanical properties, limitations for dynamic loads
11. Safety in collaborative robotics — ISO 10218 / ISO/TS 15066 in passing, e-stop and watchdog patterns

12.2 Starting keyword sets and source types

Use your institute's library / IEEE Xplore / Google Scholar. Keywords below; prioritise papers from journals like IEEE Trans. Industrial Electronics, Robotica, Mechatronics, Sensors (MDPI), and reputable conference papers (ICRA, IROS).

- Wireless: "Bluetooth vs Zigbee industrial", "BLE mesh industrial latency", "nRF24L01 reliability analysis", "2.4 GHz coexistence industrial"
- Feedback: "potentiometer vs encoder servo feedback", "closed loop control hobby servo", "DIY servo absolute position feedback"
- Control: "PID tuning servo motor", "Ziegler-Nichols manual tuning", "real-time control Arduino FreeRTOS"
- FSM / RTOS: "finite state machine embedded firmware", "FreeRTOS Arduino benchmark", "task scheduling ATmega"
- 3D printing: "FDM PLA mechanical properties robot arm", "3D printed joint durability"

A note on quality: the rubric's top band requires 'strong academic references'. Prioritise peer-reviewed papers. Hobby blogs and YouTube videos can be referenced but should be clearly flagged as 'practitioner sources' and shouldn't outweigh journal/conference citations.

13. Report Structure with Word Budget

Target: 4500 words exactly (going 10% over caps your mark at 40%). Use the IEEE conference two-column template from the IEEE author centre or Overleaf. The word counts below sum to ~4500 and map 1:1 to the rubric.

§	Section	Words	What to cover
Abstract	Abstract (single paragraph)	200	Problem, approach, core features, key quantitative results, claim of advanced features
1	Introduction	300	Industrial automation context; why wireless arms with closed-loop feedback matter; aim and objectives; paper outline
2	Literature Review	800	8 sub-topics from §12; include the wireless-protocols comparison table
3	System Design & Architecture	500	Block diagram, MCU selection justification table, wireless choice, power management overview, mode-control state diagram
4	Hardware Implementation	700	Schematics (TX, RX), sensor interfacing, power design, noise mitigation, protection, challenges faced
5	Software Development	700	FSM diagram, FreeRTOS task table, packet protocol, PID design, manual mode, autonomous mode, feedback synchronisation, code snippets
6	Robotic Arm CAD & Fabrication	400	CAD screenshots, print parameters, material justification, iteration story (before/after photos)
7	Testing & Performance Evaluation	700	Every test from §11 with a figure/table; critical discussion; optimisation loop narrative
8	Conclusion & Future Work	200	Recap of quantitative outcomes; limitations; realistic next steps
—	Acknowledgements + individual contributions table	(excluded)	Brief; contribution table does not count toward word limit
—	References (IEEE)	(excluded)	15–25 references, all cited in text

13.1 Core vs Advanced labelling — MANDATORY

The brief says: 'Students must explicitly identify and discuss which elements of their work address core requirements, and which elements represent advanced embedded systems features.' Do this with a visible convention in every technical section — a simple bolded tag works:

> *[CORE] Potentiometers on each joint provide absolute angular feedback to the microcontroller. [ADVANCED] The feedback is sampled at 100 Hz from a timer-driven ADC-complete ISR, filtered with an 8-sample moving average, and fed into a PID controller whose gains were tuned by Ziegler-Nichols (Kp = 3.2, Ki = 0.4, Kd = 0.05).*

At the end of the report, include a summary table: 'Core requirements met' (Objective 1–10 from the brief, one row each with location reference) and 'Advanced features demonstrated' (every feature from §1.1 with evidence location). This makes the marker's job easy — and easy markers give higher marks.

14. Supplementary Material (zip) Contents

The brief asks for 'a zip file containing all supplementary materials'. Structure it cleanly — markers appreciate it and it reinforces the professional-conduct angle of the teamwork mark.

- /code/transmitter/ Arduino project with source, README, build instructions
- /code/receiver/ Arduino project with FreeRTOS, PID, FSM, README
- /cad/ SolidWorks/Fusion files + STL exports + printed-parts photos
- /schematics/ KiCad source + PDF exports for transmitter and receiver
- /testing/ CSV raw logs + Python/Excel analysis scripts + generated plots
- /videos/ demo_manual.mp4, demo_autonomous.mp4, demo_safety.mp4, and a 3-minute overview
- /individual_contributions/ three named sub-folders with per-person artefacts
- /references/ PDFs of any open-access papers you cited (helps markers verify)

15. Demonstration Video Checklist

The brief says 'a demonstration video must be submitted'. A good video can rescue marks on the Hardware and Testing rows if the marker can't physically inspect everything. Cover these in order, keep to 4–6 minutes:

12. Title card: module, group member names, date
13. Wide shot: both units switched on, pairing indicator flashing
14. Manual mode: joystick → every joint moves, gripper open/close
15. Feedback demo: physically push a joint, show on a laptop screen that the pot reads the new angle and the arm self-corrects
16. Mode switch: press the button, status LED changes
17. Autonomous mode: scan → detect → classify → pick → place, repeated for 5 objects of different colours
18. Safety demo: hit the e-stop mid-motion, arm parks in safe pose
19. Loss-of-comms demo: turn off the transmitter, arm goes into SAFE within 1 s
20. Quick testbench shot: laptop screen showing latency histogram / PID plot
21. Closing: individual contributions listed on a card